SIDDHARTHA ACADEMY OF HIGHER EDUCATION

(DEEMED TO BE UNIVERSITY)

VIJAYAWADA – 520007

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

HOME ASSIGNMENT ON

VEHICLE SERVICE CENTER

FOR

OBJECT ORIENTED PROGRAMMING THROUGH JAVA

SUBMITTED BY

Roll Number	:	25EU02105
Name of Student	:	P.V.S.Bhavya Samhitha
Section	:	III-AIML B
Department	:	AI&ML

Academic Year: 2026–2027

Question
Develop a Vehicle Service Center using Vehicle, Customer and Service classes. Demonstrate overloaded methods and object references.

1. Aim

To develop a Vehicle Service Center in Java using the classes `Vehicle`, `Customer` and `Service`, demonstrating method overloading (compile-time polymorphism) and the behaviour of object references: aliasing, sharing of objects, passing references to methods, the difference between `==` and `equals()`, and `null` references.

2. Introduction

A vehicle service center repairs and services vehicles for its customers. Every vehicle has a registration number, a model and a mileage reading. Every customer owns a vehicle, and every service is a job that is booked for a customer. The cost of a service can be worked out in several ways: from the number of labour hours, from a labour amount plus the cost of spare parts, or from both of these with a discount applied for regular customers.

Instead of inventing a different method name for every way of calculating the cost, the `Service` class uses one meaningful name, `calculateCost()`, with different parameter lists. This feature is called **method overloading**, and it makes the program easier to read and to use. The classes also hold references to each other: a `Customer` refers to a `Vehicle` and a `Service` refers to a `Customer`. This makes the service center a very good example for understanding how **object references** work in Java, because several variables may refer to the same object and a change made through one of them is visible through all the others.

This is a console-based program. It is divided into three parts: Part A demonstrates the overloaded methods, Part B demonstrates object references and Part C demonstrates the null reference. The program prints the result of every step so that the behaviour of Java can be observed directly.

3. Objectives

- To create the classes `Vehicle`, `Customer` and `Service` with private (encapsulated) fields.
- To overload the method `calculateCost()` with different numbers and types of parameters.
- To show how the compiler selects an overloaded method, including type promotion (widening).
- To show that two reference variables can point to the same object (aliasing).
- To show how an object is passed to a method, i.e. that a copy of the reference is passed.
- To compare objects using `==` and `equals()` and to handle a `null` reference safely.

4. System Requirements

The program is a simple console application and needs only a basic Java development environment.

Item	Requirement
Language	Java (JDK 8 or any later version)
Compiler and runtime	javac and java commands of the JDK
Editor / IDE	Notepad, VS Code, IntelliJ IDEA, Eclipse or NetBeans
Operating system	Windows, Linux or macOS
Source file name	ServiceCenter.java (the name of the public class)
Memory	Less than 256 MB of RAM is sufficient

5. Theory and Concepts Used

5.1 Method Overloading

Overloading means having several methods with the same name in the same class but with different parameter lists. The parameter lists may differ in the number of parameters, in the types of the parameters or in the order of the types. In this program `calculateCost(int)`, `calculateCost(double, double)` and `calculateCost(double, double, int)` are three overloaded methods of the `Service` class.

5.2 Rules for Overloading

- The method name must be the same and the parameter list must be different.
- The return type alone cannot be used to distinguish two methods; changing only the return type is a compile-time error.
- Access modifiers and exceptions may be different but do not take part in the selection of the method.
- Overloaded methods may be in the same class or the child class (inherited versions are also considered).
- Constructors can also be overloaded in the same way as ordinary methods.

5.3 Compile-time Polymorphism

When an overloaded method is called, the compiler looks at the number and the static types of the arguments and decides which version will run. Because this decision is made during compilation, overloading is called compile-time (static) polymorphism or early binding. For example, `calculateCost(3)` calls the version with an `int` parameter (3 hours of labour), while `calculateCost(1200.0, 3500.0)` calls the version with two `double` parameters (labour amount and parts cost).

5.4 Type Promotion in Overloading

If no method matches the argument types exactly, Java widens the arguments in the order byte, short, int, long, float, double and searches again. The call `calculateCost(1200, 3500)` has two `int` arguments, but there is no method with two `int` parameters. The compiler therefore widens both arguments to `double` and calls `calculateCost(double, double)`. If two methods are equally suitable after widening, the call is ambiguous and the compiler reports an error.

5.5 Overloading versus Overriding

Point	Overloading	Overriding
Where	In the same class (or its subclasses)	In a subclass of the parent class
Parameters	Must be different	Must be the same
Binding	Compile time (static)	Run time (dynamic)
Decided by	Types of the arguments	Type of the actual object
Purpose	Same operation in different forms	Specialised behaviour in the child class

5.6 Object References and the Heap

A variable of a class type does not contain the object itself. The object is created on the heap using `new`, and the variable holds a reference (like an address) of that object. Primitive variables such as `int` and `double` hold the value directly, whereas variables such as `Vehicle v1` hold only a reference. This is why copying a reference does not copy the object.

5.7 Aliasing

When one reference variable is assigned to another, as in `Vehicle alias = c1.getVehicle();`, both variables point to the same object. This is called aliasing. A change made through one reference is visible through the other, because there is only one object. In the service center, two customers can share one family vehicle, and a change in its mileage is seen by both of them.

5.8 Passing Objects to Methods

Java always passes arguments by value. When an object is passed, the value that is copied is the reference. The method receives its own copy of the reference, so it can use it to change the object (the caller sees the change), as `addKilometres()` does. However, if the method assigns a new object to its parameter, as `replaceVehicle()` does, only the local copy of the reference changes and the reference in the caller is unaffected.

5.9 The == Operator and the equals() Method

The operator == compares references and is true only when both variables refer to the same object. The method `equals()` of the `Object` class behaves in the same way, but a class can override it to compare the contents. `Vehicle` overrides `equals()` so that two vehicles with the same registration number are equal, and it also overrides `hashCode()` so that equal objects have equal hash codes.

5.10 The null Reference

A reference that does not point to any object holds the special value `null`. Calling a method through a null reference throws a `NullPointerException`. The program therefore tests `vehicle == null` before using the vehicle of a customer.

5.11 Encapsulation

The fields of all three classes are private and are accessed through methods such as `getMileage()` and `setMileage()`. This protects the data from accidental changes and allows the class to control how its data is used.

6. Class Description

Class	Fields	Important methods
Vehicle	regNo, model, mileage	getRegNo(), getModel(), getMileage(), setMileage(), equals(), hashCode(), display()
Customer	id, name, vehicle (reference)	getName(), getVehicle(), display()
Service	serviceId, customer (reference), totalCost, HOURLY_RATE	calculateCost() (three overloaded versions), addToBill(), printBill()
ServiceCenter	None	main(), addKilometres(), replaceVehicle()

The relationship between the classes is as follows: a `Service` object refers to one `Customer`, a `Customer` object refers to at most one `Vehicle`, and one `Vehicle` object may be referred to by many customers.

7. Algorithm

1. Start the program.
2. Create the class `Vehicle` with regNo, model and mileage, and override `equals()` and `hashCode()`.
3. Create the class `Customer` with id, name and a reference to a `Vehicle`.
4. Create the class `Service` with a reference to a `Customer`, the constant HOURLY_RATE and the three overloaded `calculateCost()` methods.
5. In `main()`, create a `Vehicle`, a `Customer` and a `Service`.

6. Call `calculateCost()` with one `int`, two `double`, three arguments and two `int` arguments to show how the overloaded method is selected.
7. Create a second reference to the same `Vehicle`, change the mileage through it and display the mileage through the original reference.
8. Share the vehicle with a second customer and pass it to `addKilometres()` and `replaceVehicle()` to show how references behave in method calls.
9. Compare the vehicle with an equal copy using `==` and `equals()`.
10. Create a customer whose vehicle is null, handle it safely and print the bills.
11. Stop.

8. Complete Java Program

Save the source file as `ServiceCenter.java`. Compile it with `javac ServiceCenter.java` and run it with `java ServiceCenter`.

```
class Vehicle {
    private String regNo;
    private String model;
    private int mileage;

    Vehicle(String regNo, String model, int mileage) {
        this.regNo = regNo;
        this.model = model;
        this.mileage = mileage;
    }

    String getRegNo() { return regNo; }
    String getModel() { return model; }
    int getMileage() { return mileage; }
    void setMileage(int mileage) { this.mileage = mileage; }

    // equals() compares the contents (registration number), == compares references
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Vehicle)) return false;
        return regNo.equals(((Vehicle) o).regNo);
    }

    @Override
    public int hashCode() { return regNo.hashCode(); }

    void display() {
        System.out.println("Vehicle: " + model + " (" + regNo + "), Mileage: " + mileage + " km");
    }
}

class Customer {
    private int id;
    private String name;
    private Vehicle vehicle;           // object reference (may be null)
```

```
    Customer(int id, String name, Vehicle vehicle) {
        this.id = id;
        this.name = name;
        this.vehicle = vehicle;         // copies the reference, not the object
    }

    String getName() { return name; }
    Vehicle getVehicle() { return vehicle; }

    void display() {
        System.out.println("Customer " + id + ": " + name);
        if (vehicle == null) {
            System.out.println("No vehicle registered");
        } else {
            vehicle.display();
        }
    }
}

class Service {
    static final double HOURLY_RATE = 400.0;

    private int serviceId;
    private Customer customer;          // reference to a Customer object
    private double totalCost = 0;

    Service(int serviceId, Customer customer) {
        this.serviceId = serviceId;
        this.customer = customer;
    }

    // Overloaded method 1: labour given as hours (int)
    double calculateCost(int hours) {
        return hours * HOURLY_RATE;
    }

    // Overloaded method 2: labour amount and parts cost (two doubles)
    double calculateCost(double labour, double parts) {
        return labour + parts;
    }

    // Overloaded method 3: labour, parts and a discount percentage
    double calculateCost(double labour, double parts, int discountPercent) {
        return (labour + parts) * (100 - discountPercent) / 100.0;
    }

    void addToBill(double cost) {
        totalCost += cost;
    }

    void printBill() {
        System.out.println("Service ID: " + serviceId + " for " + customer.getName());
        Vehicle v = customer.getVehicle();
        if (v == null) {
            System.out.println("No vehicle to service");
            return;
```

```
        }
        System.out.println("Vehicle: " + v.getModel() + " (" + v.getRegNo() + ")");
        System.out.println("Total Bill: Rs." + totalCost);
    }
}

public class ServiceCenter {

    // The parameter holds a copy of the reference, so the object itself can be changed
    static void addKilometres(Vehicle v, int km) {
        v.setMileage(v.getMileage() + km);
    }

    // Re-assigning the parameter does not change the caller's reference
    static void replaceVehicle(Vehicle v) {
        v = new Vehicle("KA01ZZ9999", "Dummy Car", 0);
    }

    public static void main(String[] args) {
        System.out.println("VEHICLE SERVICE CENTER");

        Vehicle v1 = new Vehicle("TS09AB1234", "Honda City", 45000);
        Customer c1 = new Customer(1, "Ravi Kumar", v1);
        Service s1 = new Service(1001, c1);

        // ---- Part A: overloaded methods (resolved at compile time) ----
        System.out.println("\n--- Overloaded methods ---");
        double a = s1.calculateCost(3);                  // int            -> hours
        double b = s1.calculateCost(1200.0, 3500.0);     // double, double -> amounts
        double c = s1.calculateCost(1200.0, 3500.0, 10); // with discount
        double d = s1.calculateCost(1200, 3500);         // int args widened to double
        System.out.println("calculateCost(3)                 = Rs." + a);
        System.out.println("calculateCost(1200.0, 3500.0)    = Rs." + b);
        System.out.println("calculateCost(1200.0, 3500.0,10) = Rs." + c);
        System.out.println("calculateCost(1200, 3500)        = Rs." + d);
        s1.addToBill(c);

        // ---- Part B: object references ----
        System.out.println("\n--- Object references ---");
        Vehicle alias = c1.getVehicle();                 // second reference, same object
        System.out.println("alias == v1 : " + (alias == v1));
        alias.setMileage(45500);
        System.out.println("v1 mileage after change through alias: " + v1.getMileage());

        Customer c2 = new Customer(2, "Anita Rao", v1);  // two customers share one vehicle
        addKilometres(c2.getVehicle(), 500);
        System.out.println("c1 sees mileage: " + c1.getVehicle().getMileage());

        replaceVehicle(v1);                              // caller's reference is unchanged
        System.out.print("After replaceVehicle(): ");
        v1.display();

        Vehicle copy = new Vehicle("TS09AB1234", "Honda City", 46000);
        System.out.println("v1 == copy      : " + (v1 == copy));
        System.out.println("v1.equals(copy) : " + v1.equals(copy));

        // ---- Part C: null reference ----
```

```
        System.out.println("\n--- Null reference ---");
        Customer c3 = new Customer(3, "Walk-in Guest", null);
        c3.display();

        // ---- Bills ----
        System.out.println("\n--- Bills ---");
        s1.printBill();
        new Service(1002, c3).printBill();
    }
}
```

9. Explanation of the Program

Vehicle class

`Vehicle` stores regNo, model and mileage as private fields. `setMileage()` changes the mileage and `display()` prints the details. `equals()` returns true when the registration numbers are the same, and `hashCode()` is overridden to be consistent with `equals()`.

Customer class

`Customer` stores the id, the name and a reference to a `Vehicle`. The constructor copies the reference, not the vehicle, so the customer and the code that created the vehicle share one object. `display()` checks for null before it uses the vehicle.

Service class

`Service` holds a reference to a `Customer` and a running total. The first `calculateCost()` takes the labour hours and returns hours multiplied by the hourly rate of Rs. 400. The second takes a labour amount and the cost of parts and returns their sum. The third also takes a discount percentage and returns the sum reduced by that percentage. `printBill()` reaches the vehicle through the customer reference and prints a message if there is no vehicle.

main() method

Part A calls all the overloaded versions, including a call with `int` arguments that are widened to `double`. **Part B** creates the alias, shares the vehicle with a second customer, passes it to `addKilometres()` (which changes the object) and `replaceVehicle()` (which cannot change the caller's reference), and compares `v1` with an equal copy. **Part C** creates a customer with no vehicle (null). Finally the bills of both services are printed.

10. Sample Output

```
VEHICLE SERVICE CENTER

--- Overloaded methods ---
calculateCost(3)                 = Rs.1200.0
calculateCost(1200.0, 3500.0)    = Rs.4700.0
calculateCost(1200.0, 3500.0,10) = Rs.4230.0
```

```
calculateCost(1200, 3500)        = Rs.4700.0

--- Object references ---
alias == v1 : true
v1 mileage after change through alias: 45500
c1 sees mileage: 46000
After replaceVehicle(): Vehicle: Honda City (TS09AB1234), Mileage: 46000 km
v1 == copy      : false
v1.equals(copy) : true

--- Null reference ---
Customer 3: Walk-in Guest
No vehicle registered

--- Bills ---
Service ID: 1001 for Ravi Kumar
Vehicle: Honda City (TS09AB1234)
Total Bill: Rs.4230.0
Service ID: 1002 for Walk-in Guest
No vehicle to service
```

11. Output Explanation

- **calculateCost(3)** selects the `int` version: 3 hours × Rs. 400 = Rs. 1200.
- **calculateCost(1200.0, 3500.0)** selects the two-`double` version: 1200 + 3500 = Rs. 4700.
- **calculateCost(1200.0, 3500.0, 10)** applies a 10% discount: 4700 × 90 / 100 = Rs. 4230.
- **calculateCost(1200, 3500)** has two `int` arguments; they are widened to `double`, so the result is Rs. 4700 again.
- **alias == v1 is true** because both variables refer to one object, so setting the mileage to 45500 through `alias` is seen through `v1`.
- **c1 sees mileage 46000** because the second customer shares the same vehicle and `addKilometres()` added 500 km to that one object (45500 + 500).
- **After replaceVehicle()** the vehicle is still the Honda City, because the method re-assigned only its local copy of the reference.
- **v1 == copy is false** because they are two different objects, but **v1.equals(copy) is true** because the registration numbers are the same.
- The guest customer has a null vehicle, so the program prints "No vehicle registered" and "No vehicle to service" instead of failing with a `NullPointerException`.
- The bill of service 1001 is Rs. 4230, the discounted cost that was added to the bill.

12. Test Cases

Method call	Method selected	Result
calculateCost(3)	calculateCost(int)	Rs. 1200.0
calculateCost(1200.0, 3500.0)	calculateCost(double, double)	Rs. 4700.0

Method call	Method selected	Result
calculateCost(1200.0, 3500.0, 10)	calculateCost(double, double, int)	Rs. 4230.0
calculateCost(1200, 3500)	calculateCost(double, double) after widening	Rs. 4700.0
alias == v1	Reference comparison	true
v1 == copy	Reference comparison	false
v1.equals(copy)	Overridden equals()	true
Customer with null vehicle	Null check in display() and printBill()	Message printed, no exception

13. Advantages and Limitations

Advantages

- One method name is used for related operations, which makes the program easy to read and remember.
- New ways of calculating cost can be added by writing another overloaded method without changing the existing calls.
- Sharing one object through several references saves memory and keeps the data consistent.
- The null checks make the program safe against `NullPointerException`.

Limitations

- Overloaded methods with similar parameter types can be confusing and may cause ambiguity errors.
- Aliasing can cause unexpected changes, because a change made through one reference affects all other references.
- The program uses fixed rates and no database, so the data is lost when the program ends.

14. Possible Enhancements

- Store customers, vehicles and service history in a file or a database.
- Add different types of vehicles (two-wheeler, car, truck) using inheritance.
- Add a menu so that the user can book services and print bills interactively.
- Add GST and a loyalty discount to the billing.

15. Conclusion

The Vehicle Service Center was implemented using the classes `Vehicle`, `Customer` and `Service`. The program demonstrates method overloading with different numbers and types of parameters, type

promotion, aliasing of object references, passing objects to methods, the difference between `==` and `equals()`, and the safe handling of null references.

Result: The program calculates service costs using the overloaded methods, shows how object references behave, and prints the bills of the customers.